

DISP-Unit-Project

社内トライアル開発をすぐに始めるための、モダンなAWSアーキテクチャ共有基盤

🎯 プロジェクトの背景・目的

新しいアイデアを試したいとき、インフラ構築から始めていては時間がかかりすぎます。DISPユニットでは、**トライアル開発をすぐに始められる共通基盤**をあらかじめ用意することで、各チームが「作りたいもの」に集中できる環境を整えました。

この基盤が解決する課題は2つあります。

- 「毎回ゼロから作る」コストの削減 — 認証・API・デプロイの仕組みを再発明しなくてよい
- モダンなアーキテクチャの共通言語化 — SPA + BFF + コンテナ構成（React / Lambda / ECS）の実例をチームで共有できる

本番を見据えた設計を最初から組み込んでいるため、トライアルで作ったものをそのまま本番に近い環境へ育てることもできます。

⚡ クイックスタート

まず手を動かしたい人向けの最短手順です。フロントエンドだけならバックエンドの準備なしで起動できます。

```
git clone https://github.com/unit-developer-team/user-management.git
cd DISP-Unit-Project/frontend
npm install
cp .env.example .env # 環境変数を設定（下記「環境変数」を参照）
npm run dev          # http://localhost:5173 で起動
```

APIまで繋いだE2Eの確認や、AWSへのデプロイ手順は「[動かし方](#)」を参照してください。

👤 役割別ガイド

この基盤は、役割ごとに「始めるための手順」が異なります。それぞれの入口を以下に示します。

対象	最初にやること	詳細
フロントエンドエンジニア	<code>frontend/</code> をCloneして <code>npm run dev</code>	<code>.env.example</code> を <code>.env</code> にコピーして起動するだけで <code>localhost:5173</code> で動く。バックエンドの準備は不要
バックエンドエンジニア	<code>containers/api/</code> に処理を追加して <code>npm start</code>	認証・ルーティング・スケールは基盤側が担保済み。ロジックを書くことに集中できる

対象	最初にやること	詳細
チーム全員	mainブランチにpushする	フロントエンド・APIコンテナともに自動でビルド・デプロイされる。Lambdaのみ現時点で手動

設計思想

この基盤が大切にしている考え方は3つあります。これらの思想が、アーキテクチャの各技術選定の根拠になっています。

公開までの障壁をなくす

「作ったものをすぐ外に出せる」状態をデフォルトにします。CDN配信・HTTPS化・キャッシュ設定はあらかじめ整っており、フロントエンドエンジニアがコードを書いた翌日にはURLを共有できます。フロントとバックは独立しているため、画面だけ先に公開してAPIは後から繋ぐといった進め方もできます。

壊れない・止まらないバックエンドを前提にする

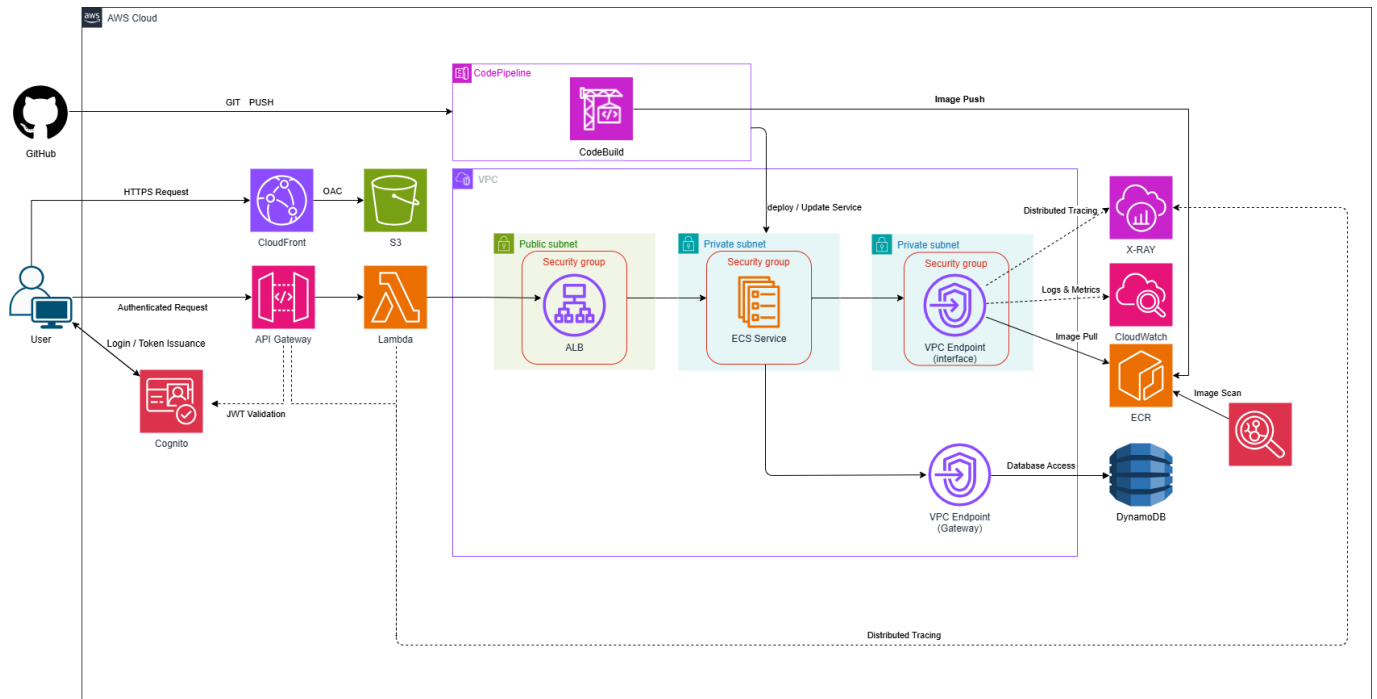
スケールや可用性の心配をアプリ開発者がしなくてよい設計にします。ECS Fargateはトラフィックに応じてコンテナ数を自動調整し、DynamoDBはレプリケーションとバックアップをマネージドで提供します。データ構造はスキーマレスのため、「試してみて、違ったら直す」を繰り返しやすいです。

pushするだけで本番に届く仕組みを標準にする

デプロイを手作業にしません。コード変更をpushすると、テスト・ビルド・デプロイが自動で走り、問題があれば自動停止します。「動いているかどうか不安なまま作業を進める」状態を避けるために、CI/CDを基盤の一部として最初から組み込んでいます。

アーキテクチャ

上記の設計思想を実現する技術構成は以下の通りです。フロントとバックの完全分離が、試行錯誤しやすい設計上の根拠になっています。



```

[ブラウザ]
  ↓ HTTPS
[CloudFront] ← S3 (Reactビルド成果物)
  ↓
[Cognito] ← 認証 (サインアップ・サインイン)
  ↓
[API Gateway] ← Cognito Authorizer
  ↓
[Lambda BFF] ← 振り分け・バリデーション
  ↓ HTTP
[ALB]
  ↓
[ECS Fargate (APIコンテナ)] ← Express / TypeScript
  ↓ VPCエンドポイント経由 (インターネットを通らない)
[DynamoDB (UsersTable)]

```

ブラウザからのリクエストはCloudFrontで受け、Cognito認証を通過した後、Lambda BFFが内容を検証してECSコンテナに転送します。ECSからDynamoDBへの通信はVPCエンドポイント経由のため、インターネットを経由しません。

構成の選択理由

構成要素	トライアル開発における意図
S3 + CloudFront	静的サイト配信のベストプラクティス。フロントの公開・更新が素早くできる
ECS Fargate	サーバー管理不要。コンテナを差し替えるだけでアプリを更新できる
Lambda BFF	入口の共通化と、各チームの実装の自由度を両立する
DynamoDB	スキーマレスで素早くデータ構造を変えられる。トライアル期間中の変更に強い

構成要素

トライアル開発における意図

Cognito	認証基盤を自前で作らなくてよい。セキュアな認証をすぐに組み込める
VPCエンドポイント	インターネットを経由せずにAWSサービスと通信。セキュリティを高める

セキュリティの考え方

トライアル用途としてシンプルさと安全性のバランスを取っています。「外部からの通信を最小化し、内部通信にも認証を設ける」という方針で設計されています。

- **外部通信の制御** — ブラウザとの通信はCloudFront経由のHTTPSのみ。ECS↔DynamoDB間はVPCエンドポイントを使いインターネットを通らない
- **API認証** — すべてのAPIリクエストにCognitoのIDトークンが必要。API GatewayのCognito Authorizerが検証する
- **内部通信の認証** — Lambda↔ECS間はINTERNAL_TOKENによるトークン認証を行う。外部のAPIキーとは別に管理し、万が一の漏洩時の影響範囲を限定する
- **最小権限** — 各コンポーネントのIAMロールは必要なリソースのみアクセス可能な権限に絞っている

なお、現時点ではALBがパブリックサブネットに配置されており、Internal ALB + LambdaのVPC配置は今後の強化項目として残っています（ロードマップ参照）。



ローカル開発

前提条件

- Node.js 20以上
- AWS CLIの設定済み
- Gitのインストール済み

Step 1: まずローカルで動かして全体感を掴む

フロントエンドだけでも動作確認できます。最初はここから始めることを推奨します。

```
cd frontend
npm install
cp .env.example .env # 環境変数を設定（下記参照）
npm run dev          # http://localhost:5173 で起動
```

Step 2: バックエンドも繋いで、APIの疎通を確認する

フロントとバックエンドを両方ローカルで動かし、E2Eの動作を確認します。

```
# APIコンテナを起動
cd containers/api
npm install
```

```
npm run build
npm start
```

Lambda BFFはビルドのみ（デプロイは手動）：

```
cd lambda/bff
npm install
npm run build
```

Step 3: AWSにデプロイして、チームに共有できる状態にする

mainブランチにpushするだけで自動デプロイされます（詳細は「デプロイ方法」を参照）。

技術スタック

フロントエンド

技術	役割
React 18 / TypeScript	UIフレームワーク
Vite	ビルドツール
AWS Amplify	Cognito連携・認証管理
Vitest / Testing Library	ユニットテスト
S3	静的ファイルのホスティング
CloudFront	CDN配信・HTTPS化

バックエンド

技術	役割
Lambda（Node.js 20.x / TypeScript）	BFF（振り分け・バリデーション）
ECS Fargate / Express / TypeScript	APIコンテナ
ALB	ロードバランシング
DynamoDB	ユーザーデータの永続化
API Gateway	エンドポイント管理・認証
Cognito	認証基盤（サインアップ・サインイン）
SQS	非同期処理（将来実装予定）

インフラ・運用

技術	役割
IAM	最小権限によるアクセス制御
VPC / Subnet / Security Group	ネットワーク閉域化
VPCエンドポイント	インターネットを経由しないAWSサービス通信
CloudWatch	ログ収集・監視
CodePipeline / CodeBuild	CI/CDパイプライン
ECR	Dockerイメージの管理

環境変数・設定ファイル

frontend/.env

ローカル開発時に必要です。GitHubには含まれません（`.gitignore`で除外済み）。

```
VITE_API_URL=https://xxxxxxxxx.execute-api.ap-northeast-1.amazonaws.com/prod
VITE_USER_POOL_ID=ap-northeast-1-xxxxxxxxx
VITE_USER_POOL_CLIENT_ID=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

AWS CodeBuildの環境変数（本番環境）

サービス	キー	説明
フロントエンド	<code>VITE_API_URL</code>	API GatewayのURL
フロントエンド	<code>VITE_USER_POOL_ID</code>	CognitoユーザープールID
フロントエンド	<code>VITE_USER_POOL_CLIENT_ID</code>	CognitoアプリケーションクライアントID
フロントエンド	<code>CLOUDFRONT_DISTRIBUTION_ID</code>	CloudFrontのディストリビューションID
APIコンテナ	<code>AWS_ACCOUNT_ID</code>	AWSアカウントID

Lambda環境変数

キー	説明
<code>ALB_URL</code>	ALBのDNS名（ <code>http://</code> から始まる）
<code>INTERNAL_TOKEN</code>	内部通信用トークン（Lambda⇄ECS間の認証）

ECSタスク定義の環境変数

キー	説明
<code>TABLE_NAME</code>	DynamoDBのテーブル名（ <code>UsersTable</code> ）

キー	説明
INTERNAL_TOKEN	内部通信用トークン（Lambdaと同じ値）
PORT	Expressのポート番号（3000）

API仕様

概要

項目	内容
ベースURL	https://xxxxxxxxxx.execute-api.ap-northeast-1.amazonaws.com/prod
データ形式	JSON
認証方式	Cognito IDトークン（Bearer認証）

全てのエンドポイントにCognitoのIDトークンが必要です:

```
Authorization: Bearer {CognitoIDトークン}
```

エンドポイント一覧

メソッド	パス	説明
GET	/users	ユーザー一覧取得
POST	/users	ユーザー登録
GET	/users/{id}	ユーザー詳細取得
DELETE	/users/{id}	ユーザー削除

リクエスト・レスポンス例

ユーザー登録（POST /users）

```
// リクエスト
{
  "name": "山田太郎",
  "email": "yamada@example.com"
}

// レスポンス（201 Created）
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "山田太郎",
  "email": "yamada@example.com",
```

```
"createdAt": "2024-01-01T00:00:00.000Z"
}
```

エラーハンドリング

ステータスコード	説明
400	リクエストボディが不正
401	認証トークンがない・無効
404	ユーザーが存在しない
409	同じIDのユーザーが既に存在する
500	サーバー内部エラー

詳細は [API仕様書](#) を参照してください。



デプロイ方法

フロントエンドとAPIコンテナは、mainブランチへのpushで自動デプロイされます。Lambdaのみ現時点では手動デプロイが必要です（CI/CD化は今後の課題）。

フロントエンド

```
git add .
git commit -m "変更内容"
git push origin main
# → CodePipelineが自動起動 → テスト → ビルド → S3デプロイ → CloudFrontキャッシュ無効化
```

APIコンテナ

```
git add .
git commit -m "変更内容"
git push origin main
# → CodePipelineが自動起動 → Dockerビルド → ECRプッシュ → ECSローリングアップデート
```

Lambda（手動）

```
cd lambda/bff
npm run build
cd dist
# PowerShell
```



```
Compress-Archive -Path * -DestinationPath ../function.zip -Force
# AWSコンソールからfunction.zipをアップロード
```

📁 ディレクトリ構成

```
DISP-Unit-Project/
├── frontend/                ← Reactアプリ
│   ├── src/
│   │   ├── api/            ← API呼び出しロジック
│   │   ├── pages/         ← 画面コンポーネント
│   │   └── tests/         ← テスト
│   ├── buildspec-frontend.yml
│   └── package.json
├── lambda/
│   └── bff/                ← Lambda BFF
│       └── src/
│           ├── handlers/   ← Lambdaエントリーポイント
│           ├── services/   ← ECSへの転送ロジック
│           └── utils/      ← バリデーション・レスポンス
├── containers/
│   └── api/                ← ECS APIコンテナ
│       └── src/
│           ├── routes/     ← ルーティング
│           ├── controllers/ ← リクエスト処理
│           ├── services/   ← DynamoDBアクセス
│           └── models/     ← 型定義
├── docs/
│   ├── frontend.md        ← フロントエンド技術詳細
│   ├── backend.md         ← バックエンド技術詳細
│   └── api-spec.md        ← API仕様書
├── buildspec-api.yml      ← APIコンテナのCI/CD設定
└── README.md
```

📄 ドキュメント

ドキュメント	内容
フロントエンド技術詳細	React・Amplify・CloudFront・CI/CDの詳細
バックエンド技術詳細	Lambda・ECS・DynamoDB・VPC・IAMの詳細
API仕様書	エンドポイント・リクエスト・エラー仕様

コストと運用負荷

マネージドサービス中心の構成のため、インフラの運用負荷はほぼ発生しません。スケールアップ・バックアップ・冗長化はAWS側が自動で担保します。

コストはすべて従量課金です。小規模なトライアル用途（開発メンバー数名・リクエスト数が少ない期間）では月数百円～数千円程度に収まることが多いです。ECS Fargateは起動中のコンテナ数に比例するため、不要なタスクを停止しておくことで抑制できます。

ロードマップ

現在は**トライアル開発の最小構成**として動作する状態です。複数チームが本格的にこの基盤を使い始める次のフェーズに向けて、以下の優先順位で拡張を進めます。

優先度の判断基準は「現時点で開発者の手作業が残っているか」と「データ量・利用チーム数が増えたときに最初に限界が来るか」の2軸で決めています。

優先度: 高 — 現状、手作業またはテスト空白が残っている領域

- LambdaのCI/CDパイプライン構築 — 現在手動のデプロイを自動化し、フロントエンド・バックエンドと同様のフローに統一する

優先度: 中 — 利用規模が拡大したときに最初に限界が来る領域

- ECS Workerコンテナの実装（SQSによる非同期処理） — 複数チームが同じ基盤を使い始めたときの負荷分散と、重い処理の切り離しを実現する
- ページネーション対応（DynamoDB ScanからQueryへの変更） — データ量が増えても安定したレスポンスを維持するために必要

優先度: 中 — 本番移行を見据えた運用成熟度の向上

- IaC（TerraformまたはCDK）によるインフラのコード化 — 現在は手動管理の部分を残したまま。インフラの再現性・レビュー可能性を高める
- 本番環境向けのセキュリティ強化（Internal ALB + LambdaのVPC配置） — より本番に近い構成への移行。現在はトライアル用途のため一部をシンプルに保っている

現時点でカバーしない範囲

本基盤はトライアル開発フェーズに特化した構成であり、以下は現時点で対象外となっています。本番サービスへの移行を検討する際は、別途要件定義が必要になります。

- **高可用性・障害復旧** — マルチリージョン構成やRTOを定めた障害復旧設計は含まない
- **高度なセキュリティ制御** — WAFによるリクエストフィルタリング、セキュリティ監査ログの長期保管は対象外
- **コスト最適化** — リザーブドインスタンスやSavings Planによる料金最適化は行っていない
- **本番運用の観測性** — アラート設計・ダッシュボード・オンコール体制などの運用フローは含まない